

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	192424390
Name	Venu G

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 40%

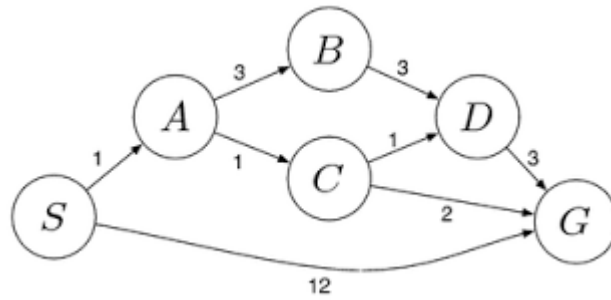
QUESTIONS: 5

Total Marks: 50

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path

from the starting warehouse **S** to the destination warehouse **G** using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I _Venu G_ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

Question 1: Water Jug Problem

Problem: Given a 4-gallon jug and a 3-gallon jug with no measuring markings, and a pump to fill the jugs, obtain exactly 2 gallons of water in the 4-gallon jug. Assumptions: a jug can be filled from the pump; water can be poured onto the ground; water can be poured from one jug to another; no other measuring devices are available.

Problem Formulation

- State: (x, y) where x = water in the 4-gallon jug (0–4), y = water in the 3-gallon jug (0–3)
- Initial State: $(0, 0)$
- Goal State: $(2, y)$ for any value of y
- Actions: Fill A, Fill B, Empty A, Empty B, Pour $A \rightarrow B$, Pour $B \rightarrow A$
- Path Cost: 1 per action (minimize number of actions)

Solution Sequence

Step	Action	4-gal Jug (A)	3-gal Jug (B)
0	Start	0	0
1	Fill B	0	3
2	Pour $B \rightarrow A$	3	0
3	Fill B	3	3
4	Pour $B \rightarrow A$ (A takes only 1 more)	4	2
5	Empty A	0	2
6	Pour $B \rightarrow A$	2	0

Result: After Step 6, Jug A (4-gallon) contains exactly 2 gallons of water.

Question 2: Mars Rover Agent Analysis

1. Percepts

- Camera images (stereo/panoramic, hazard cameras)
- Spectrometer / chemical composition readings of soil and rocks
- GPS / positional and inertial navigation data
- Terrain slope, obstacle-distance, and dust readings
- Temperature and atmospheric sensor data
- Battery / solar power level
- Wheel odometry and motor feedback
- Commands / signals received from Earth or orbiter relay

2. Operating Environment Characterization

Property	Type	Reason
Observability	Partially observable	Sensors have limited range; dust and terrain hide information
Determinism	Stochastic	Unpredictable terrain, communication delay, dust storms
Episodic / Sequential	Sequential	Current movement/decisions affect future state
Static / Dynamic	Semi-dynamic	Environment mostly static but conditions change over time
Discrete / Continuous	Continuous	Terrain, movement, sensor readings are continuous
Single / Multi-agent	Single-agent (mostly)	One rover acts independently; no adversary
Known / Unknown	Unknown	Martian terrain is largely unexplored

3. Actions

- Move forward / backward, turn, steer around obstacles
- Extend / operate robotic arm, drill or scoop soil/rock samples
- Activate camera, spectrometer, or other instruments
- Transmit collected data to orbiter/Earth
- Recharge via solar panel positioning

- Enter safe/sleep mode to conserve power

4. Performance Evaluation

- Distance traveled safely (without damage)
- Number and quality of samples successfully collected and analyzed
- Accuracy and completeness of transmitted scientific data
- Power/energy efficiency
- Number of obstacles avoided vs. collisions
- Mission time and operational longevity
- Successful two-way communication rate with Earth

5. Most Suitable Agent Architecture

A model-based, goal/utility-based agent is most suitable.

- Since the environment is partially observable, the rover must maintain an internal model/state of the world (terrain map, past readings) — a simple reflex agent cannot cope with this.
- Since the rover has multiple competing objectives (maximize scientific value, conserve power, ensure safety, meet deadlines), it needs a utility function to choose the best action among several goal-satisfying options.
- Hence: model-based utility-based agent, often augmented with learning capability to improve terrain-traversal and hazard-avoidance decisions over time.

Question 3: 8-Queens Problem — Search Space & Formulation

Problem Formulation (Incremental Formulation)

- States: Any arrangement of 0 to 8 queens on the board, at most one queen per column (placed left to right)
- Initial State: Empty board (no queens placed)
- Actions: Add a queen to the leftmost empty column, in any row not attacked by an existing queen
- Transition Model: Returns the board with the new queen added
- Goal Test: 8 queens placed with no two queens attacking each other (same row, column, or diagonal)
- Path Cost: Not significant — only the final state matters

Search Space Size

- Naive (any square, any number of queens): $C(64,8) \approx 4.4 \times 10^9$
- With one-queen-per-column restriction: $8^8 \approx 16.7$ million
- With incremental placement + immediate conflict checking: reduces to only a few thousand nodes explored in practice

Applicable Search Strategies

Strategy	How It Applies
Depth-First Search with Backtracking	Place a queen column by column; if a placement leads to no valid position in a later column, backtrack to the previous column and try the next row. Classic and most efficient complete-search approach.
Breadth-First Search	Explores all partial boards level by level (column by column) — correct but memory-heavy since branching factor is 8.
Local Search (Hill-Climbing / Min-Conflicts)	Start with all 8 queens randomly placed (one per column). Repeatedly move the queen with most conflicts to the row minimizing conflicts until 0 conflicts remain. Very fast in practice.

Example Backtracking Trace (Partial)

Queen 1 → column 1, row 1. Queen 2 → column 2, must avoid row 1 and diagonals → row 3. Queen 3 → column 3, avoid rows/diagonals of Q1, Q2 → try row 5. Continue until either all 8 placed validly (a solution) or a dead end forces backtracking to an earlier column.

Question 4: OLA Cab Booking — Agent Type & Pseudocode

Type of Problem-Solving Agent

A Goal-Based (Problem-Solving) Agent operating over a search-based route/decision model. It formulates a goal (reach the destination), formulates the problem (available routes and cab options), searches for the best solution (route + cab type), and executes it. Since it must weigh cost vs. time vs. comfort across multiple cab types, it also functions as a utility-based agent at the decision-selection stage.

Pseudocode

```
function BOOK_CAB_AGENT(pickup_location, destination, user_preferences):
```

```
    // 1. Formulate Goal
```

```
    goal <- REACH(destination) with acceptable cost/comfort/time
```

```
    // 2. Formulate Problem
```

```
    problem <- {
```

```
        initial_state: pickup_location,
```

```
        actions: possible_routes(pickup_location, destination),
```

```
        goal_test: state == destination,
```

```
        path_cost: function(distance, traffic, cab_type)
```

```
    }
```

```
    // 3. Search candidate routes and cab options
```

```
    cab_types <- GET_AVAILABLE_CAB_TYPES(pickup_location)
```

```
    options <- empty list
```

```
    for each cab in cab_types:
```

```
        route <- SEARCH(problem)           // e.g., UCS/A* over road network
```

```
        fare <- CALCULATE_FARE(route, cab)
```

```
        eta <- CALCULATE_ETA(route, cab, traffic_data)
```

```
        utility <- UTILITY(fare, eta, comfort_level(cab), user_preferences)
```

```
        options.append({cab, route, fare, eta, utility})
```

```
    // 4. Select best option based on utility (or let user choose)
```

```
    best_option <- ARGMAX(options, key = utility)
```

```
DISPLAY(options)
```

```
selected <- USER_SELECT(options) or best_option
```

```
// 5. Execute
```

```
driver <- ASSIGN_NEAREST_DRIVER(selected.cab, pickup_location)
```

```
CONFIRM_BOOKING(driver, pickup_location, destination)
```

```
NAVIGATE(driver, route = selected.route)
```

```
return trip_confirmation
```

Question 5: Uniform Cost Search — Logistics Network

Sample Weighted Delivery Network (Start = S, Goal = G)

S --2--> A
S --5--> B
A --1--> B
A --7--> C
B --2--> D
D --1--> C
C --3--> G
D --6--> G

UCS Execution (Priority Queue, lowest cumulative cost expanded first)

Step	Node Expanded	Cumulative Cost	Frontier (node: cost)
1	S	0	A:2, B:5
2	A	2	B:3 (replaces 5), C:9
3	B	3	D:5, C:9
4	D	5	C:6 (replaces 9), G:11
5	C	6	G:9 (replaces 11)
6	G	9	— Goal reached

Least-Cost Path: $S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow G$

Total Cost = $2 + 1 + 2 + 1 + 3 = 9$

Why UCS works here: UCS always expands the node with the lowest path cost $g(n)$ from the frontier (a priority queue), guaranteeing the optimal (least-cost) path is found before the goal is expanded, as long as all edge costs are non-negative — which holds in this logistics network of fuel/time costs.

